

MODULE 04 · 25 MIN

Reading code together

Claude Code 101 · Beginner Workshop · Module 4 of 8

The fastest way to understand someone else's code is to read it with a patient partner.
Today Claude is that partner.

What you'll learn

By the end of this 25-minute lesson you will be able to:

- 1 Hand Claude a file from your project and ask for a plain-English explanation.
- 2 Ask follow-ups about specific lines or functions.
- 3 Spot the limits — when Claude is guessing and you need to read the source yourself.

Why this matters

- Half of every job is reading code you did not write. The other half is reading code you wrote six months ago and forgot.
- Stack Overflow answers explain patterns; Claude can explain **your** file with **your** names and **your** structure.
- This is the lowest-stakes way to build trust in Claude's answers. Reading is not editing — nothing breaks if Claude is wrong.

The one concept

Claude reads what you paste, then talks about it.

The session has no automatic access to your filesystem unless you give it permission later (Module 07). For now, you copy the file's contents into the prompt and Claude explains what it sees. That is all.

Because Claude only sees what you paste, the quality of the explanation depends on you pasting the right amount: too little = guessing, too much = surface skim.



Show me

A real conversation about a small Bash script:

```
$ claude
> Explain this script for someone who has never used Bash. Three short paragraphs.

#!/usr/bin/env bash
set -euo pipefail
for f in *.log; do
    lines=$(wc -l < "$f")
    echo "$f has $lines lines"
done
```

This script processes every file in the current folder whose name ends in `*.log`. For each one it counts the lines and prints a one-line summary.

The first two lines tell Bash to be strict: stop on the first error, treat undefined variables as errors, and fail a pipeline if any step fails.

The loop body uses `wc -l < "$f"` (rather than `wc -l "$f"`) so the filename is not echoed alongside the count, which makes the final `echo` line easier to control.

> What does `set -euo pipefail` actually catch?

It catches: (1) any command that exits non-zero (`-e`), (2) any reference to a variable that was never set (`-u`), and (3) a non-zero exit anywhere in a pipeline, not just the last command (`pipefail`). Together these turn "silently broken" into "stop loudly".

► Try it yourself

Open Claude Code, paste the script from `exercises/beginner/part-04/starter/`, and ask for a plain-English explanation. Then ask one follow-up about a line you didn't understand.

Save the whole transcript into `~/code-read.txt`. The exercise README walks you through it.

Time budget: 10 minutes.

Common mistakes

- **Pasting 800 lines and saying "explain this".** Claude will skim and miss the important parts. Paste 30–80 lines at a time.
- **Believing every detail.** Claude does not run the code. If it says "this returns 7", verify before you act on it.
- **Forgetting that variable names matter.** If a file uses cryptic names like `x` and `tmp`, Claude's explanation will also be vague. That's a smell to fix in the file, not in the prompt.
- **Asking "is this good code?" too early.** That's Module 03 (Asking for what you want) territory. In this module, just ask what it does.

✓ Lesson reflection

Take 60 seconds:

1. Did Claude's explanation match what the code actually does? How sure are you?
2. Was there a line where Claude was clearly guessing? How could you tell?
3. Would you trust Claude's reading of a 30-line script? A 300-line one? A 3000-line one?

What's next →

Module 05 — **Editing one file safely** — finally lets Claude propose changes. You'll learn how to accept, reject, and undo so nothing surprises you.

Budget for Module 05: 30 minutes.

Glossary card

- **Project context:** Information about your project (often in CLAUDE.md) that you give Claude so its answers fit your codebase.
- **Prompt:** The text you send to Claude. One message in the conversation.